Cupcake

Aplicação web acadêmica para uma loja de cupcakes, desenvolvida como parte de um projeto de Engenharia de Software na Universidade Cruzeiro do Sul (UNICSUL), na disciplina PIT-II.

O projeto disponibiliza uma API REST e uma interface web para gerenciamento de produtos e pedidos, utilizando **Java 21**, **Spring Boot 4.1.2** e **PostgreSQL**.

1. TECNOLOGIAS

Tecnologia	Finalidade
`Java 21`	Linguagem de programação
`Spring Boot 4.1.2`	Framework da aplicação
`Spring Web MVC`	API REST
`Spring Data JPA`	Persistência de dados
`Hibernate`	Mapeamento objeto-relacional
`PostgreSQL`	Banco de dados principal
`Flyway`	Migrações do banco de dados
`Maven Wrapper`	Construção e gerenciamento de dependências
`JUnit 6` / `Mockito`	Testes automatizados e isolamento de dependências
`Testcontainers` / `H2`	Testes de integração com PostgreSQL / BD em memória
`HTML5`, `CSS3`, `JS ES Modules`	Estrutura, apresentação e organização do frontend
`Git`	Controle de versão

2. ARQUITETURA

A aplicação utiliza uma arquitetura de **Monólito Modular (Modular Monolith) combinada com Arquitetura em Camadas (Layered Architecture)**.

Backend: Organizado em três módulos principais (`product`, `order` e `shared`). Cada módulo funcional separa suas responsabilidades em *Controller*, *DTO*, *Entity*, *Repository* e *Service*. Utiliza DTOs na fronteira da API REST, evitando a exposição direta das entidades JPA.

Frontend: Aplicação independente desenvolvida com HTML, CSS e JavaScript puro (Vanilla JavaScript), organizada em módulos ES. A comunicação com o backend ocorre via API REST.

3. FUNCIONALIDADES IMPLEMENTADAS

Produtos

- Cadastro de produtos.
- Consulta de produtos ativos e por ID.
- Atualização e desativação de produtos.
- Listagem paginada e ordenação.

Frontend

- Catálogo e consulta de detalhes de produtos.
- Carrinho de compras local (navegador).
- Atualização de quantidades e remoção de itens.
- Checkout, criação e consulta de pedidos.
- Navegação via rotas Hash e integração REST.

Pedidos

- Criação e consulta de pedidos por ID.
- Listagem paginada de pedidos.
- Cálculo de subtotais e valor total.
- Preservação de nomes/preços no ato da compra.
- Consolidação de IDs repetidos na requisição.

Tratamento de Erros

- Tratamento centralizado de exceções.
- Respostas HTTP utilizando Problem Details.
- Validação de requisições e regras de negócio.

4. ESTRUTURA DO PROJETO

```
cupcake-unicsul/
├── .mvn/wrapper/
├── docs/uml/
├── frontend/
│   ├── assets/images/
│   ├── css/
│   └── js/ (api/, components/, pages/, services/, utils/)
│   └── index.html
├── src/
│   ├── main/ (java/, resources/)
│   └── test/ (java/, resources/)
├── pom.xml | README.md | LEIAME.md | mvnw
```

5. PRÉ-REQUISITOS

- JDK Java 21
- PostgreSQL
- Docker (para os testes de integração com Testcontainers)
- *O projeto possui Maven Wrapper embutido.*

6. CONFIGURAÇÃO DO BANCO DE DADOS

Endereço padrão: `jdbc:postgresql://localhost:5432/cupcake`

```
export DB_USERNAME=cupcake_user
export DB_PASSWORD='sua_senha'
# Opcional: export DB_URL='jdbc:postgresql://localhost:5432/cupcake'
```

Crie o banco de dados e configure o usuário PostgreSQL antes de iniciar. O Flyway gerencia automaticamente as migrações durante a inicialização.

7. EXECUTANDO A APLICAÇÃO

Inicie o backend no perfil de desenvolvimento:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

Ou execute a partir do JAR gerado (`./mvnw clean package`):

```
java -jar target/cupcake-0.0.1-SNAPSHOT.jar --spring.profiles.active=dev
```

Backend disponível em `http://localhost:8080`. O frontend deve ser servido por um servidor HTTP na porta 5500.

8. EXECUTANDO OS TESTES

```
./mvnw clean test                          # Suíte completa
./mvnw test -Dtest=ProductTest             # Classe específica
```

9. API REST

Caminho-base: `/api/v1`

Método	Endpoint	Descrição
`GET`	`/products`	Listar produtos ativos
`GET`	`/products/{id}`	Consultar um produto
`POST`	`/products`	Cadastrar um produto
`PUT`	`/products/{id}`	Atualizar um produto
`DELETE`	`/products/{id}`	Desativar um produto
`GET`	`/orders`	Listar pedidos
`GET`	`/orders/{id}`	Consultar um pedido
`POST`	`/orders`	Criar um pedido

10. DOCUMENTAÇÃO

Documentos disponíveis em `docs/`: `arquitetura.md`, `api.md`, `banco-de-dados.md`, `config-ambiente-desenvolvimento.md`, e `testes.md`.

11. LIMITAÇÕES ATUAIS & 12. PROJETO ACADÊMICO

A implementação não contempla: autenticação/autorização, integração com pagamentos, upload de imagens, carrinho persistente no backend, gerenciamento de status de pedidos ou cancelamento. É um projeto acadêmico voltado à aplicação prática de conceitos de Engenharia de Software.

Autor: Lussac Prestes Maia — GitHub (@lussac7) | LinkedIn